

Riempire DropDown:

```
def fill_ddshape(self):
    shape=self._model.getAllShapes()
    shapeDDOption= list(map(lambda x: ft.dropdown.Option(data=x, key=x,
on_click=self._choiceDDShape), shape))
    self._view.ddshape.options = shapeDDOption
    self._view.update_page()
```

```
def _choiceDDShape(self, e):
    self._shapeValue = e.control.data
```

```
def getGraphDetails(self):
    return len(self._graph.nodes), len(self._graph.edges)
```

ARCO PESO MAGGIORE

```
def getArchiPesoMaggiore(self):
    archi = []
    for u, v, peso in self._graph.edges(data=True):
        archi.append((u, v, peso["weight"]))
    # Archi sono Attore1, Attore2, Peso dell'arco

    archi.sort(key=lambda x: x[2], reverse=True) # Ordiniamo in base al peso
    return archi[:5]
```

```
def get5bestEdges(self):
    edges=[]
    for e in self._graph.edges(data=True):
        edges.append(e)
    edges.sort(key=lambda x: x[2]['weight'], reverse=True)
    return edges[:5]
```

```
➔ Controller:
for a in archiPesoMaggiore:
    self._view.txt_result.controls.append(ft.Text(f"{a[0].driverRef} -> {a[1].driverRef} ({a[2]}"))
```

MIGLIORI 5 NODI:

```
def get5bestNodes(self):
    nodes=[]
    for n in self._graph.nodes:
        nodes.append((n, self._graph.degree(n))) #Tupla con nodo e grado corrispondente
    nodes.sort(key=lambda x: x[1], reverse=True)
    return nodes[:5]
```

Nodi più profittevoli:

```
def getNodiPiuProfittevoli(self):
    listNodesPesata=[]
    for n in self._graph.nodes:
        score=0
        for e in self._graph.out_edges(n, data=True): #Per ogni arco uscente --> sommiamo a score il
            peso dell'arco (è una tupla con p1, p2, peso)
            score+=e[2]["weight"]
        for e in self._graph.in_edges(n, data=True): #Per ogni arco entrante --> sottraiamo a score il peso
            dell'arco
            score-=e[2]["weight"]
        listNodesPesata.append((n,score))
    listNodesPesata.sort(key=lambda x: x[1], reverse=True) #Ordiniamola per lo score x[1]
    return listNodesPesata[:5]
```

VICINI DI UN NODO (con relativo peso degli archi)

```
def getVicini(self, teamCode):
    source=self._codeT[teamCode]
    vicini=self._graph.neighbors(source)
    viciniTuples=[]
    for v in vicini:
        peso_arco = self._graph[source][v]["weight"]
        #Lista di tuple con vicino e peso dell'arco che ti porta
        viciniTuples.append((v, peso_arco))

    viciniTuples.sort(key=lambda x: x[1], reverse=True)
    return source, viciniTuples
```

COMPONENTE CONNESSA

```
def getComponenteConnessa(self):
    #conn=nx.node_connected_component(self._graph, source) #Richieste grafo --> nodo di partenza.
    #Noi vogliamo, però, la componente connessa del grafo
    conn = nx.connected_components(self._graph) #E' un set di nodi
    return list(conn)
```

MAX COMPONENTE CONNESSA

```
def getMaxCompConnessa(self):
    return max(nx.connected_components(self._graph), key=len)
```

➔ Grado di ogni nodo:

```
for c in maxCompConnessaOrdinata:
    grado = self._model._graph.degree(c)
```

Componete connessa massima con peso dei nodi ordinati in senso decrescente

```
def getMaxCompConn(self):
    compConn = max(nx.connected_components(self._graph), key=len)
    maxCompConn = []
    for c in compConn:
        pesoMax=0
        for n in self._graph.neighbors(c):
            peso=self._graph[c][n]['weight']
            if peso > pesoMax:
                pesoMax=peso
        maxCompConn.append((c, pesoMax))

    maxCompConn.sort(key=lambda x: x[1], reverse=True)
    return maxCompConn
```

Componente debolmente connessa:

```
def getCompDebolmenteConnesse(self):
    compConn=list(nx.weakly_connected_components(self._graph))
    return len(compConn)
```

sum(cast(replace(replace(m.worlwide_gross_income, '\$', ''),',', '')) as unsigned)) as peso.

- ➔ Il primo replace sostituisce '\$' con ''
- ➔ Il secondo replace sostituisce , con ''
- ➔ Il cast forza la stringa in un intero, senza segno (unsigned)

m.worlwide_gross_income like '\$%' ➔ considera solo le righe che iniziano così.

Per arrotondare numeri float: round(numero, n. cifre decimali)

def _hasNode(self, idOggetto):

```
    return idOggetto in self._idMapOb #Dobbiamo vedere le chiavi del dizionario (il for se dobbiamo
    ciclare sui valori)
```

RICORSIONE:

```
def getBestPath(self):
    self._bestPath=[]
    self._bestScore=0

    parziale=[]
    for n in self._graph.nodes():
        parziale=[n]
        self._ricorsione(parziale)

    return self._bestPath, self._bestScore

def _ricorsione(self, parziale):
    #Condizione di ottimalità:
    if len(parziale)> len(self._bestPath):
        score=self._getScore(parziale)
        if score> self._bestScore:
            self._bestScore=score
            self._bestPath=parziale

#2) Caso ricorsivo:
for n in self._graph.neighbors(parziale[-1]):
    if n in parziale:
        continue

    densita=n.Population/n.Area
    if densita > parziale[-1].Population/parziale[-1].Area:
        parziale.append(n)
        self._ricorsione(parziale)
        parziale.pop()
```

```

def _getScore(self, path):
    sum_pesi=0
    sum_dist = 0
    for i in range(len(path) - 1):
        sum_pesi += self._graph[path[i]][path[i + 1]]["weight"]
        sum_dist += path[i].distance_HV(path[i + 1])

    if sum_dist == 0:
        return 0

    score=sum_pesi/sum_dist
    return score

```

Prendere archi con peso strettamente crescente:

```

for n in self._graph.neighbors(parziale[-1]):
    self._graph[parziale[-2]][parziale[-1]]["weight"] > self._graph[parziale[-1]][n]["weight"]:

```

```

def getBestLung(self):
    self._bestPath=[]
    self._bestLun=0
    nodi=list(self._graph.nodes)
    nodi.sort(key=lambda x: x.GeneID)

    parziale=[]
    listaNodiValida=[]
    for n in nodi:
        if n.Essential!="?":
            listaNodiValida.append(n)

    for n in listaNodiValida:
        parziale.append(n)
        self._ricorsione(parziale, 0, listaNodiValida)
        parziale.pop()
    return self._bestPath, self._bestLun

def _ricorsione(self, parziale, index, listaCandidati):
    #1) Caso terminale:
    if len(parziale) > self._bestLun:
        self._bestLun=len(parziale)
        self._bestPath=copy.deepcopy(parziale)

    #A parità di lunghezza, controlliamo le componenti connesse
    # --> creiamo un sottografo contenete solo i nodi correnti
    sub_attuale=self._graph.subgraph(parziale)
    num_comp_attuale=nx.number_connected_components(sub_attuale)

```

```

sub_best=self._graph.subgraph(self._bestPath)
num_comp_best = nx.number_connected_components(sub_best)

# Il testo richiede che il numero di componenti connesse sia MINIMO
if num_comp_attuale < num_comp_best:
    self._bestPath = copy.deepcopy(parziale)

#3) Caso ricorsivo --> esploriamo i nodi rimanenti
for i in range(index, len(listaCandidati)):
    nodo_corrente=listaCandidati[i]

    if nodo_corrente not in parziale:
        parziale.append(nodo_corrente)
        self._ricorsione(parziale, i+1, listaCandidati)
        parziale.pop()

```

Nodi devono far parte di due componenti connesse distinte:

```

def _ricorsione(self, parziale, k, nodiValidi, index ):
    #1) Caso Terminale: abbiamo trovato esattamente k piloti
    if len(parziale)==k:
        scarto=self._calcola_scarto_piloti(parziale)
        if scarto < self._getBestScarto:
            self._getBestScarto=scarto
            self._getBestList=copy.deepcopy(parziale)
        return

    #2)Caso Ricorsivo: cerchiamo di aggiungere un nuovo pilota
    for i in range(index, len(nodiValidi)):
        nuovo_pilota=nodiValidi[i]

        #Verifichiamo che il pilota rispetti il vincolo delle componenti distinte:
        valido=True
        for p in parziale:
            #Se esiste un cammino tra il nuovo pilota e uno già in pista, stoppiamo!
            if nx.has_path(self._graph, p, nuovo_pilota): #--> Metodo di NetworkX che controlla se c'è un
                qualsiasi
                    # collegamento tra due nodi
                    #(se True, appartengono alla stessa componena connessa)

            valido=False
            break

        if valido:
            parziale.append(nuovo_pilota)
            self._ricorsione(parziale, k, nodiValidi, i+1)
            parziale.pop()

```

python

Vicini uscenti (grafo orientato)

self._graph.neighbors(nodo) # oppure

self._graph.successors(nodo)

Vicini entranti

self._graph.predecessors(nodo)

Peso di un arco

self._graph[u][v]["weight"]

Archi con dati

self._graph.edges(data=True) # $\rightarrow (u, v, \{ "weight": \dots \})$

self._graph.out_edges(nodo, data=True)

self._graph.in_edges(nodo, data=True)

TEST MODEL:

from model.model import Model

myModel=Model()

myModel.buildGraph(7)

nodi, archi=myModel.getGraphDetails()

print("Grafo creato")

print(f"Numero nodi: {nodi}, numero archi: {archi}")

print(myModel.getArtistaMaxInfluenza())

print(myModel.getArchiPesoMaggiore())

- **Quanti brani ha acquistato ogni cliente?**

```
SELECT inv.CustomerId, SUM(il.Quantity) as totBrani
FROM invoice inv, invoiceline il
WHERE inv.InvoiceId = il.InvoiceId
GROUP BY inv.CustomerId
```

- **Quanto ha speso ogni cliente?**

-- Modo 1: usando il Total già calcolato in invoice

```
SELECT CustomerId, SUM(Total) as spesa
FROM invoice
GROUP BY CustomerId
```

-- Modo 2: ricalcolando da invoiceline (più preciso)

```
SELECT inv.CustomerId, SUM(il.UnitPrice * il.Quantity) as spesa (incasso totale)
FROM invoice inv, invoiceline il
WHERE inv.InvoiceId = il.InvoiceId
GROUP BY inv.CustomerId
```

- **Quante volte è stato acquistato ogni brano?**

```
SELECT t.TrackId, t.Name, SUM(il.Quantity) as vendite
FROM track t, invoiceline il
WHERE t.TrackId = il.TrackId
GROUP BY t.TrackId, t.Name
ORDER BY vendite DESC
```

- **Incasso per genere**

```
SELECT g.Name, SUM(il.UnitPrice * il.Quantity) as incasso
FROM genre g, track t, invoiceline il
WHERE g.GenreId = t.GenreId
AND t.TrackId = il.TrackId
GROUP BY g.Name
ORDER BY incasso DESC
```

- **Clienti che hanno acquistato brani di un certo artista**

```
SELECT DISTINCT inv.CustomerId
FROM invoice inv, invoiceline il, track t, album ab
WHERE inv.InvoiceId = il.InvoiceId
AND il.TrackId = t.TrackId
AND t.AlbumId = ab.AlbumId
AND ab.ArtistId = ?
```

count(distinct TrackId) → numero di brani in comune

count(distinct CustomerId) → numero di clienti in comune

Cliente → invoice → invoiceline → track → album → artist → genere

- **Clienti che hanno speso più della media:**

```
SELECT CustomerId, SUM(Total) as spesa
FROM invoice
GROUP BY CustomerId
HAVING SUM(Total) > (
    SELECT AVG(spesaMedia) FROM (
        SELECT SUM(Total) as spesaMedia
        FROM invoice
        GROUP BY CustomerId
    ) sub
)
```

- Per ogni artista: brani totali nel catalogo E brani venduti:

```
SELECT a.Name,
       catalogo.totCatalogo,
       vendite.totVenduto
FROM artist a,
     (SELECT ab.ArtistId, COUNT(t.TrackId) as totCatalogo
      FROM album ab, track t
      WHERE ab.AlbumId = t.AlbumId
      GROUP BY ab.ArtistId) catalogo,
     (SELECT ab.ArtistId, SUM(il.Quantity) as totVenduto
      FROM album ab, track t, invoiceline il
      WHERE ab.AlbumId = t.AlbumId
      AND il.TrackId = t.TrackId
      GROUP BY ab.ArtistId) vendite
WHERE a.ArtistId = catalogo.ArtistId
AND a.ArtistId = vendite.ArtistId
```

2. Nodi: clienti che hanno comprato almeno un brano di un genere

```
SELECT DISTINCT c.*
FROM customer c, invoice i, invoiceline il, track t
WHERE c.CustomerId = i.CustomerId
AND i.InvoiceId = il.InvoiceId
AND il.TrackId = t.TrackId
AND t.GenreId = %s
ORDER BY c.LastName, c.FirstName;
```

4. Archi: coppie di artisti comprati dallo stesso cliente

```
SELECT DISTINCT ab1.ArtistId AS a1, ab2.ArtistId AS a2
FROM invoice i1, invoice i2,
     invoiceline il1, invoiceline il2,
     track t1, track t2,
     album ab1, album ab2
WHERE i1.CustomerId = i2.CustomerId
AND i1.InvoiceId = il1.InvoiceId
```

```

AND i2.InvoiceId = il2.InvoiceId
AND il1.TrackId = t1.TrackId
AND il2.TrackId = t2.TrackId
AND t1.AlbumId = ab1.AlbumId
AND t2.AlbumId = ab2.AlbumId
AND t1.GenreId = %s
AND t2.GenreId = %s
AND ab1.ArtistId < ab2.ArtistId;

```

5. Archi pesati: coppie di artisti con numero clienti in comune

```

SELECT ab1.ArtistId AS a1,
       ab2.ArtistId AS a2,
       COUNT(DISTINCT i1.CustomerId) AS peso
FROM invoice i1, invoice i2,
     invoiceline il1, invoiceline il2,
     track t1, track t2,
     album ab1, album ab2
WHERE i1.CustomerId = i2.CustomerId
AND i1.InvoiceId = il1.InvoiceId
AND i2.InvoiceId = il2.InvoiceId
AND il1.TrackId = t1.TrackId
AND il2.TrackId = t2.TrackId
AND t1.AlbumId = ab1.AlbumId
AND t2.AlbumId = ab2.AlbumId
AND ab1.ArtistId < ab2.ArtistId
GROUP BY ab1.ArtistId, ab2.ArtistId;

```

6. Archi: coppie di clienti che hanno comprato lo stesso artista

```

SELECT DISTINCT i1.CustomerId AS c1,
               i2.CustomerId AS c2
FROM invoice i1, invoice i2,
     invoiceline il1, invoiceline il2,
     track t1, track t2,
     album ab1, album ab2
WHERE i1.InvoiceId = il1.InvoiceId
AND i2.InvoiceId = il2.InvoiceId
AND il1.TrackId = t1.TrackId
AND il2.TrackId = t2.TrackId
AND t1.AlbumId = ab1.AlbumId
AND t2.AlbumId = ab2.AlbumId
AND ab1.ArtistId = ab2.ArtistId
AND i1.CustomerId < i2.CustomerId;

```

7. Archi pesati: clienti con numero artisti in comune

```

SELECT i1.CustomerId AS c1,
       i2.CustomerId AS c2,
       COUNT(DISTINCT ab1.ArtistId) AS peso
FROM invoice i1, invoice i2,
     invoiceline il1, invoiceline il2,
     track t1, track t2,
     album ab1, album ab2
WHERE i1.CustomerId = i2.CustomerId
AND i1.InvoiceId = il1.InvoiceId
AND i2.InvoiceId = il2.InvoiceId
AND il1.TrackId = t1.TrackId
AND il2.TrackId = t2.TrackId
AND t1.AlbumId = ab1.AlbumId
AND t2.AlbumId = ab2.AlbumId
AND ab1.ArtistId < ab2.ArtistId
GROUP BY i1.CustomerId, i2.CustomerId;

```

```

    album ab1, album ab2
WHERE i1.InvoiceId = i2.InvoiceId
    AND i2.InvoiceId = i2.InvoiceId
    AND i1.TrackId = t1.TrackId
    AND i2.TrackId = t2.TrackId
    AND t1.AlbumId = ab1.AlbumId
    AND t2.AlbumId = ab2.AlbumId
    AND ab1.ArtistId = ab2.ArtistId
    AND i1.CustomerId < i2.CustomerId
GROUP BY i1.CustomerId, i2.CustomerId;

```

8. Archi: coppie di brani nella stessa playlist

```

SELECT DISTINCT pt1.TrackId AS t1,
               pt2.TrackId AS t2
FROM playlisttrack pt1, playlisttrack pt2
WHERE pt1.PlaylistId = pt2.PlaylistId
    AND pt1.TrackId < pt2.TrackId;

```

9. Archi pesati: brani con numero playlist in comune

```

SELECT pt1.TrackId AS t1,
       pt2.TrackId AS t2,
       COUNT(DISTINCT pt1.PlaylistId) AS peso
FROM playlisttrack pt1, playlisttrack pt2
WHERE pt1.PlaylistId = pt2.PlaylistId
    AND pt1.TrackId < pt2.TrackId
GROUP BY pt1.TrackId, pt2.TrackId;

```

10. Popolarità artisti

```

SELECT ab.ArtistId AS artista,
       SUM(il.Quantity) AS popolarita
FROM album ab, track t, invoiceline il
WHERE ab.AlbumId = t.AlbumId
    AND t.TrackId = il.TrackId
GROUP BY ab.ArtistId;

```

11. Incasso per artista

```

SELECT ab.ArtistId AS artista,
       SUM(il.Quantity * il.UnitPrice) AS incasso
FROM album ab, track t, invoiceline il
WHERE ab.AlbumId = t.AlbumId
    AND t.TrackId = il.TrackId
GROUP BY ab.ArtistId;

```